언리얼 엔진 스터디 5주차

매니저를 통한 게임 디자인 실습

GameInstance · GameMode · GameState · PlayerController · PlayerState · HUD

복습 1 — PlayerController & PlayerState

PlayerController — 조종의 주체

- **입력을 받아 폰을 조종하는 클래스**
  - Possess(Pawn) 으로 특정 폰에 빙의
  - UnPossess() 로 해제, 교체 시 자동 UnPossess
- **한 컨트롤러는 한 폰만 소유**
  - GetPawn() → 현재 빙의된 폰 반환
- **폰보다 먼저, 더 오래 산다**
  - 캐릭터가 죽어도 컨트롤러는 유지
  - UI 생성 / 입력 바인딩의 정석 위치

PlayerState — 데이터의 주체

- **플레이어 고유 데이터 저장소**
  - HP, 점수, 레벨, 경험치 등
  - PlayerController · Pawn 모두에서 접근
- **Pawn이 파괴돼도 살아남는다**
  - 리스폰 후에도 스탯이 영속됨
  - 그래서 ResetHP() 를 명시적으로 호출
- **GameState 와의 차이**
  - GameState: 게임 전체 (웨이브·시간 등)
  - PlayerState: 개별 플레이어 전용

핵심 **PlayerCharacter ≠ Player. 캐릭터는 사라져도 Controller + State 가 곧 플레이어다.**

복습 2 — Enhanced Input & 스프린트

Enhanced Input — 3가지 핵심 에셋

InputAction (IA_*)

"무엇"을 하는지

IA_Move / IA_Jump / IA_Sprint

InputMappingContext

키 ↔ 액션 매핑

IMC_Default: WASD→Move, Shift→Sprint

InputModifier

값 변환

Negate / Swizzle / DeadZone

스프린트 구현 흐름 (실습 1)

- IA_Sprint 생성 (Value Type: Digital) → IMC_Default 에 LeftShift 매핑
- Character 에 SprintSpeed / WalkSpeed 변수 + Sprint() / StopSprint() 함수
  - Sprint: GetCharacterMovement()->MaxWalkSpeed = SprintSpeed;
  - StopSprint: GetCharacterMovement()->MaxWalkSpeed = WalkSpeed;
- BindAction 트리거 — Started: 누르는 순간 / Completed: 떼는 순간

복습 3 — HP 바 UI + 데미지 & 리스폰

UMG — HP 바 위젯

- WBP_PlayerHUD (UserWidget) 생성
  - Canvas → VerticalBox → Text + ProgressBar
  - HPBar 의 Is Variable 체크 필수
- Percent 바인딩 함수
  - GetPlayerState → Cast<BP_PlayerState>
  - CurrentHP / MaxHP 반환 (Float)
- PlayerController 가 위젯을 생성
  - CreateWidget + AddToViewport
  - 캐릭터 죽어도 UI 는 살아있어야 함

데미지 & 리스폰 — 데이터 흐름

Timer (1s) → Character::OnTickDamage

▼

PlayerState::ApplyDamage(10)

▼

HP = 0 → IsAlive() = false

▼

Character::Destroy + ScheduleRespawn

▼

0.5s 후 → Possess(NewPawn) + ResetHP()

Possess 만 하고 ResetHP 를 빼먹으면 — 새 캐릭터가 즉시 또 죽는다 (HP 가 이미 0).

이번 주의 질문 (한 줄로)

> **"지금 이 데이터는 누구의 책임인가?"**

- 4주차까지는 PC·PS·HUD를 직접 썼다
- **이번 주는 한 발 물러서서 6개 매니저의 협력 구조를 본다**
- 다음 주에 픽업 아이템이, 9주차에 적이 등장한다.

오늘의 목표

- 6개 핵심 매니저의 역할을 구분해 설명할 수 있다
- 임의의 데이터가 주어졌을 때 "어디에 두어야 하는가"를 판단할 수 있다
- 룰(고정값)과 상태(변동값)의 분리 원칙을 적용할 수 있다
- 배포된 프로젝트에서 6개 매니저가 실제로 어떻게 협력하는지 다이어그램으로 확인할 수 있다

이번 주차 진행 방식 — 기존과 다릅니다

- 5주차는 개념적 분리가 핵심 — "어디에 두는가"라는 사고의 틀이 핵심
- 이론 (~50분) — 6개 매니저 + 룰/상태 분리 원칙
- 퀴즈 (~20분) — 데이터 배치 워크숍 (5지선다 7문제)
- 실습 (~30분) — 배포 파일 분석 + 다이어그램으로 구조 확인 (코딩 X)

넘어가기 전에 — 지금까지 배웠던 친구들

4주차까지 우리가 만났던 클래스들을 한 번 떠올려보자:

- Pawn / Character — 월드에 존재하는 "몸"
- PlayerController — 입력을 받아 폰을 조종하는 주체
- PlayerState — 폰이 죽어도 살아남는 데이터 저장소 (HP)
- UMG Widget — 화면에 그리는 친구 (HP 바)

그런데... 이게 전부일까?
게임이 커지면 더 필요한 게 생긴다.

캐릭터가 다 들고 있던 짐을, 6명이 나눠 든다

GameInstance NEW
앱 평생 사는 친구 (앱 시작~종료)

GameMode NEW
룰을 정하는 친구 (서버만)

GameState NEW
모두가 알아야 할 상태 (복제됨)

PlayerController
입력·UI·빙의 (4주차 친구)

PlayerState
내 점수·HP (4주차 친구)

HUD/Widget
화면에 그리기 (4주차 친구)

★ *절반은 이미 만났다. 오늘은 나머지 셋(GameInstance · GameMode · GameState)을 채운다.*

HUD vs Widget — 잠깐, 이 둘은 뭐가 다른가?

- **UMG Widget (UUserWidget) — 우리가 4주차에 만든 WBP_PlayerHUD**
  - 디자이너가 그리는 화면 요소 하나하나 (Progress Bar, Text 등)
  - 데이터를 소유하지 않고, PS·GS에서 읽어서 그리기만 함
- **HUD (AHUD) — PlayerController가 소유하는 화면 그리기 책임자**
  - 여러 Widget을 만들고 화면에 붙이는 관리자 역할
  - 디버그 라인 그리기 같은 저수준 그리기도 가능
- **이 강의에서는? — 거의 Widget만 쓴다. HUD 클래스는 PC가 대신 관리.**

한 줄 요약: **Widget = 그림판 한 장, HUD = 그림판들을 관리하는 액자**

6개 매니저 한눈에 보기

매니저	수명	위치	역할
GameInstance	앱 시작~종료	서버+클라	레벨 전환에도 유지되는 전역 데이터
GameMode	레벨 단위	서버 전용	게임 룰, 승리/패배 판정
GameState	레벨 단위	서버+모든 클라	모두가 알아야 할 상태
PlayerController	접속 동안	본인 클라+서버	입력, Pawn 빙의, UI 소유
PlayerState	접속 동안	서버+모든 클라	HP, 점수, 보유 아이템
HUD/UMG	접속 동안	본인 클라	화면에 그리기

★ *4주차에 본 PC·PS·Widget이 이미 있었다. 오늘은 나머지 셋(GI·GM·GS)을 채운다.*

GameInstance — 싱글톤

- **수명: 앱 시작 ~ 앱 종료**
  - 레벨이 바뀌어도 죽지 않음
  - *PIE 재시작하면 새로 생성됨 (실 빌드와 다름)*
- **무엇을 넣나**
  - 누적 플레이 횟수, 최고 점수, 플레이 타임 등, “게임을 꺼도 유지되어야 하는 정보들!”

- **무엇을 넣지 않나**
  - 너무 많이 넣으면 -> 계속 살아있어야 하는 오브젝트가 너무 많은 책임, 규모를 가지게 됨.
  - 게임 인스턴스 용량 : 게임의 ‘기본 용량’ 이라는 것을 기억할 것!

	Mobile	PC
	30GB	70GB
	34GB	84GB
	57GB	119GB
	45GB	125GB
	24GB	45GB

★ *수명이 긴 객체의 용량은, 곧 게임 전체의 용량이다. 싱글톤 객체에 너무 많이 위임하지 말 것.*

GameMode — 룰을 정하는 친구

- 수명: 레벨 단위 (레벨 바뀌면 새로 생성)
- 위치: 서버 전용 (멀티플레이에서 클라이언트엔 없음)
- 무엇을 넣나
  - 매니저 클래스 매핑 (GameStateClass·PlayerControllerClass·PlayerStateClass — 생성자에서 직접)
  - 게임의 룰
    - 승리 / 패배 조건
    - 스테이지 전체 정보 등..
  - 무엇을 넣지 않나?
    - 서버 – 클라이언트 모두가 알아야 하는 정보
    - 실시간 게임 상태.

★ *보드게임 룰북. 게임이 시작되면 바뀌지 않는 것들이 여기에 산다.*

GameState — 모두가 알아야 할 상태

- **수명: 레벨 단위**
- **위치: 서버+모든 클라이언트 (자동 복제)**
- **무엇을 넣나**
  - 현재 웨이브, 남은 시간, 스테이지 번호
  - *"내 것"이 아니라 "판 전체의 것"*
- **멀티플레이에서 모두의 화면에 똑같이 떠야 하는 건 여기**

★ *야구장 전광판. 모든 관중이 같은 숫자를 본다.*

PC vs PS — 4주차 복습 + 이번 주 추가 책임

	PlayerController	PlayerState
4주차	입력, UI, Possess	HP, 데미지
5주차 추가	**HandlePlayerDeath() (외부 진입점) + ScheduleRespawn (private 보조)**	**RemainingRespawns (BeginPlay에서 GM->MaxRespawnCount 복사)**
위치	본인 클라+서버	서버+모든 클라(복제)

★ *4주차에 Character가 죽음 처리를 하던 게 → 이제 PC가 책임진다 (이유는 잠시 후 Q6에서 다룬다)*

룰 vs 상태

이번 주의 진짜 핵심 — 같은 개념도 둘로 쪼갠다

룰 vs 상태 — 두 얼굴의 데이터

	룰 (변하지 않음)	상태 (실시간 변화)
위치	**GameMode**	**GameState / PlayerState**
예시	`MaxRespawnCount = 3`	`RemainingRespawns = 2`
누가 바꾸나	디자이너가 디테일 패널에서	게임 진행에 따라 자동으로

핵심 통찰

같은 "부활 횟수"라도

"고정 한도"는 GameMode, ***"현재 상황"은 PlayerState***

룰/상태 분리 — 왜 이게 중요한가

- **만약 룰과 상태를 섞어둔다면?**
  - MaxRespawnCount를 PlayerState에 넣었다고 치자
  - → 디자이너가 BP_GameMode 디테일 패널에서 만질 수 없음
  - → 플레이어마다 다른 룰을 가질 수 있는 어색한 구조
  - *→ "근데 왜 룰이 사람마다 다르지?"*

> *축구 경기인데 사람마다 오프사이드 규칙이 다르다? — 그게 룰을 PS에 넣은 모양새.*

데이터 배치 결정 트리

한 번에 결정 못 하겠으면 위에서부터 묻기:

1. 레벨이 바뀌어도 살아야 하나?	→	GameInstance
2. 룰 판정인가?	→	GameMode
3. 다른 플레이어도 봐야 하나?	→	GameState / PlayerState
4. 입력·UI·내 클라이언트만?	→	PlayerController

라이프사이클 — 매니저들의 등장 순서

```
앱 시작
  ↓ GameInstance 생성 (한 번만)
레벨 로드 → UWorld
  ↓ GameMode 생성 (서버)
  ↓ GameState 생성 (복제)
플레이어 접속
  ↓ PlayerController 생성
  ↓ PlayerState 생성 (복제)
  ↓ Pawn 스폰 후 Possess
  ↓ HUD/Widget 생성
```

★ *위에서부터 차례로 생성된다 — 의존성이 자연스럽게 흐른다.*

데이터를 실제로 배치해보자!

풀이 원칙 (퀴즈 시작 전)

3가지만 묻고 가자:

1

누가 봐야 하는가?

(나만? 모두?)

2

룰인가, 상태인가?

(고정값? 변동값?)

3

레벨이 바뀌어도 살아야 하는가?

(휘발? 영속?)

5지선다지만 답을 맞추는 게 목적이 아님
왜 그렇게 답했는지 설명할 수 있어야 함

Q1.

누적 플레이 횟수 / 누적 플레이 타임 — 앱을 껐다 켜도 유지되어야 함

- 변수 예시: int32 TotalPlayCount, float TotalPlayTimeSeconds
- 사용처: 메인 메뉴에서 "플레이 100시간 돌파!" 업적 표시
- 언제 증가하나: 게임 시작 시 +1, Tick으로 시간 누적

① GameInstance

② GameMode

③ GameState

④ PlayerState

⑤ PlayerController

(다음 슬라이드에서 정답 공개)

① GameInstance

- 레벨이 바뀌거나 새 게임을 시작해도 살아남아야 함
- GameMode/GameState는 레벨마다 새로 생성됨 → 부적합
- *실 빌드에서는 PIE와 다르게 앱 종료까지 살아있음*
- *영구 저장(껐다 켜도)은 별도 — USaveGame 클래스로 디스크에 저장*

Q2.

죽을 수 있는 최대 횟수 (MaxRespawnCount = 3)

- 변수 예시: int32 MaxRespawnCount
- 사용처: 게임 시작 시 한 번 정해지고, 디자이너가 밸런싱할 때 만짐
- *언제 변하나: 변하지 않는다 (그래서 "룰")*

① GameInstance

② GameMode

③ GameState

④ PlayerState

⑤ PlayerController

(다음 슬라이드에서 정답 공개)

② GameMode

- 디자이너가 "이 게임의 룰"로 정해두는 고정값
- 디테일 패널에서 0/3/999로 바꿔보며 게임이 어떻게 달라지는지 실험 가능

★ 키워드 "Max" = 룰 → GameMode

Q3.

현재 남은 부활 횟수 (RemainingRespawns = 2)

- 변수 예시: int32 RemainingRespawns
- 사용처: HUD에 "남은 목숨: 2" 표시, 0이 되면 게임 오버
- *언제 변하나: 죽을 때마다 -1*

① GameInstance

② GameMode

③ GameState

④ PlayerState

⑤ PlayerController

(다음 슬라이드에서 정답 공개)

④ PlayerState

- 게임 진행에 따라 변하는 "내 상태"
- 다른 플레이어도 볼 수 있어야 하므로 복제되는 PlayerState
- **Q2와의 짝: Max~ 는 룰(GameMode), Remaining~ 은 상태(PlayerState)**

★ *같은 개념도 두 매니저에 나뉘어 산다.*

Q4.

현재 웨이브 번호 + 다음 웨이브까지 남은 시간

- 변수 예시: int32 CurrentWave, float TimeUntilNextWave
- 사용처: 모든 플레이어 HUD 상단에 똑같이 표시
- *언제 변하나: 서버가 Tick으로 카운트다운, 0 되면 다음 웨이브*

① GameInstance

② GameMode

③ GameState

④ PlayerState

⑤ PlayerController

(다음 슬라이드에서 정답 공개)

③ GameState

- "모두가 알아야 할 상태"
- 내 것이 아니라 판 전체의 상태 → PlayerState 아님
- *멀티플레이라면 모든 클라이언트에 자동 복제*

Q5.

마우스 감도 설정 (이 클라이언트에서만)

- 변수 예시: float MouseSensitivity = 1.0f
- 사용처: 옵션 메뉴에서 슬라이더로 조정, 카메라 회전 시 곱연산
- *언제 변하나: 옵션에서 사용자가 바꿀 때만*

① GameInstance

② GameMode

③ GameState

④ PlayerState

⑤ PlayerController

(다음 슬라이드에서 정답 공개)

⑤ PlayerController

- 입력 처리는 PC의 영역
- 다른 플레이어가 알 필요 없는, 본인 클라이언트에만 존재하는 데이터

★ *Enhanced Input의 IMC Modifier에서도 감도 조절 가능 — 4주차 복습.*

Q6.

죽었을 때 부활 결정 + Pawn 파괴 책임

- 함수 예시: void HandlePlayerDeath()
- 사용처: HP=0 감지 → Pawn Destroy → 0.5초 후 새 Pawn 스폰 → Possess
- 누가 호출하나: 누군가 한 명이 책임지고 처리해야 함

① Character가 직접

② GameMode

③ GameState

④ PlayerState

⑤ PlayerController

이게 가장 헷갈린다. 잠시 생각해보자.

⑤ PlayerController

- *"캐릭터가 자기 자신의 죽음을 결정하고 파괴하는 게 맞는가?"*
- Pawn은 빙의되었다 풀렸다 하는 "몸"
- **빙의를 관리하는 주체는 PC → 죽음 처리도 PC가 갖는 게 자연스럽다**

★ *4주차의 Character::OnTickDamage가 죽음까지 처리하던 건 임시 — 5주차에 이사 간다.*

Q7.

이번 판에서 처치한 적의 수 (KillCount)

- 변수 예시: int32 KillCount = 0
- 사용처: 점수판 UI에 모든 플레이어의 킬 수 나란히 표시
- *언제 변하나: 적을 죽일 때마다 +1, 게임 종료 시 정산*

① GameInstance

② GameMode

③ GameState

④ PlayerState

⑤ PlayerController

(다음 슬라이드에서 정답 공개)

④ PlayerState

- "이번 판에서" → GameInstance 아님
- 다른 플레이어도 점수판에서 보고 싶을 데이터 → 복제되는 PlayerState

함정 토론: PC에 두면 무슨 일이 생길까?

(힌트: 멀티플레이에서 다른 플레이어의 PC는 내 클라이언트에 없다)

퀴즈 정리 — 7문제로 본 핵심 패턴

데이터 성격	어디에
앱 평생 (껐다 켜도)	**GameInstance**
룰 (Max, 한도)	**GameMode**
판 전체 상태 (Wave, Time)	**GameState**
내 상태 (HP, Kill, Respawns)	**PlayerState**
입력·UI·내 Pawn 빙의	**PlayerController**

★ *이 표를 외우는 게 아니라 결정 트리를 체화하는 게 목표.*

실습 — 배포 파일로 구조 확인 (~30분)

Step 0 배포 파일 받기

- **Week5_ManagerStructure.zip (수업 시간에 공유)**
- 4주차 프로젝트에 GameInstance/GameState/GameMode 분리
- + 죽음 처리 책임 재배치까지 적용된 상태

★ *Visual Studio에서 솔루션 열기 → 빌드 한 번 → 에디터 실행*

Step 1 매니저 6개 둘러보기 (10분)

각 클래스 파일의 UPROPERTY 선언만 본다. 코드 작성이 아님!

매니저	확인할 파일	들어있는 데이터
GameInstance	`StudyGameInstance.h`	TotalPlayCount, BestStageReached
GameMode	`StudyGameMode.h`	MaxRespawnCount (룰)
GameState	`StudyGameState.h`	CurrentStage, CurrentWave, TimeUntilNextWave
PlayerState	`StudyPlayerState.h`	MaxHP, CurrentHP, RemainingRespawns
PlayerController	`StudyPlayerController.h`	HandlePlayerDeath() (public 진입점), ScheduleRespawn / RespawnCharacter (private 보조)
HUD/Widget	`WBP_PlayerHUD`	PS의 HP·Respawns + GS의 Stage·Wave·Time + GI의 TotalPlayCount 읽어 표시

★ *퀴즈 정답과 실제 코드 위치를 대조해본다.*

Step 1 체크포인트

- ✅ 각 헤더 파일을 열어 UPROPERTY 선언을 봤다
- ✅ Q1~Q7 정답이 실제로 그 위치에 있는 걸 확인했다
- ✅ "내가 푼 답이 정말 맞았나?" — 한 번 더 자문해봤다

Step 2 매니저 협력 다이어그램 (10분)

GameInstance (앱 평생)

TotalPlayCount: 3
BestStageReached: 5

↓

GameMode (서버만) + GameState (복제)

MaxRespawnCount + 매니저 클래스 매핑(GS/PC/PS) / CurrentStage, CurrentWave, TimeUntilNextWave

↓

PlayerController + PlayerState

입력·UI·HandlePlayerDeath / HP, RemainingRespawns

↓

Character + HUD

Character: "죽었다"만 PC에 알림 / HUD: PS·GS를 읽어 화면에 그림

Step 2 — 다이어그램 확인 포인트

- ✓ MaxRespawnCount(룰)와 RemainingRespawns(상태)가 다른 매니저에 있다
- ✓ HUD는 데이터를 소유하지 않고 읽기만 한다 — 그래서 PS, GS를 모두 참조한다
- ✓ Character는 더 이상 자기 죽음을 결정하지 않는다 — PC에 알리기만 함
- ✓ 4주차의 5가지 책임 폭탄이 6개 매니저로 분산됐다

룰 → 상태 복사 — Q2와 Q3을 잇는 코드 한 줄

동작

- StudyPlayerState::BeginPlay 에서 GM의 룰을 자기 상태로 복사한다
- `World->GetAuthGameMode<AStudyGameMode>()->MaxRespawnCount → RemainingRespawns`

의미

- Q2(MaxRespawnCount는 GameMode)와 Q3(RemainingRespawns는 PlayerState)의 "짝"이 코드 한 줄로 표현된다
- 룰은 한 곳(GameMode)에서 정의 → 게임 시작 시 각 PlayerState로 복사 → 이후 PS는 자기 상태로 차감

실험

- World Settings → GameMode 디테일 패널 → MaxRespawnCount를 0/3/999로 변경
- PIE 시작 직후 HUD의 RemainingRespawns가 그 값으로 시작하는지 눈으로 확인
- 0이면 첫 사망에 게임 오버 / 999면 사실상 무제한 부활

매니저별 추가 결과 — 5주차 종료 시점

매니저	5주차 추가	9주차에 채울 자리
GameInstance	TotalPlayCount, BestStageReached	*(이미 다 작동)*
GameState	CurrentStage, CurrentWave, TimeUntilNextWave	*EnemiesRemaining*
GameMode	MaxRespawnCount + 매니저 클래스 매핑(GS·PC·PS Class)	*CurrentStageDifficulty, AdvanceStage()*
PlayerState	RemainingRespawns	*KillCount*
PlayerController	HandlePlayerDeath()	*(충분)*

★ *빈 자리가 미리 보인다 — 9주차에 적이 등장할 때 이 표를 다시 본다.*

다음 떡밥 — "스테이지별 설정은 어디에?"

스테이지가 10개 있다고 치자. 각 스테이지마다:

- 적 등장 패턴
- 웨이브 수, 웨이브별 적 수
- 보스 출현 시점
- BGM, 배경 색상

Poor한 방법

GameMode에 if (stage==1) … else if (stage==2) … 떡칠

→ 스테이지 추가할 때마다 코드 수정 + 재빌드

→ 디자이너가 손도 못 댐

더 나은 방법

엑셀 같은 테이블에 저장해두고 GameMode가 읽어서 적용

→ DataTable / DataAsset (다음 주 이후에 다룸!)

룰을 코드에서 데이터로 옮기면 게임이 확장 가능해진다.

핵심 질문 (가져갈 것)

- **MaxRespawnCount를 PlayerState에 둔다면 무엇이 어색해질까?**
  - *(힌트: 디자이너가 디테일 패널에서 어떻게 만질까)*
- **GameInstance에 너무 많이 넣으면 위험한 이유?**
  - *(힌트: 2주차의 God Object)*
- **Tick에서 카운트다운하는 GameState — 멀티플레이라면 어디에서 돌려야 할까?**
  - *(힌트: 서버만)*

> ***정답을 외우는 게 아니라 이런 질문을 스스로 던질 수 있느냐가 핵심.***

오늘 배운 것

- **6개 매니저 — GameInstance / GameMode / GameState / PC / PS / HUD**
- **데이터 배치 4가지 결정 기준 — 누가 보나, 룰/상태, 수명, 위치**
- **룰 vs 상태 — 같은 개념도 둘로 쪼개진다**

- 다음 주(6주차): 이 구조 위에 첫 픽업 아이템(체력 포션) + C++ 매크로 시스템

이번 주 한 줄 요약: **"지금 이 데이터는 누구의 책임인가?"**

오늘 완성한 기능들 — 배포 파일에서 직접 돌려볼 수 있는 동작

세션 누적

✓ TotalPlayCount 증가 — StudyGameInstance::Init / PIE 재시작마다 +1

웨이브 자동 진행

✓ 30초 카운트다운 → CurrentWave++ → 리셋 — StudyGameState::Tick

부활 시스템

✓ MaxRespawnCount 디자이너 튜닝 — World Settings → GameMode 디테일 패널 → 0/3/999
✓ 룰 → 상태 복사 — PS::BeginPlay에서 GM->MaxRespawnCount 읽어 RemainingRespawns에 복사
✓ 사망마다 RemainingRespawns -1 — PS::DecrementRespawn
✓ 0.5초 뒤 자동 부활 + HP 100% 복구 — PC::ScheduleRespawn → RespawnCharacter
✓ 게임 오버 분기 — RemainingRespawns=0 시 입력 차단 + Output Log "Game Over"

죽음 처리 책임 이전

✓ Character.cpp 의 OnTickDamage 에 Destroy() 호출이 "없다" — PC가 책임짐

★ 4주차의 한 자리 책임이 6개 매니저에 자기 자리를 찾아 들어갔다 — 기능이 늘어난 게 아니라 "자리가 잡힌" 주차

정리 & Q&A

다음 주 예고

- **6주차 — 첫 픽업 아이템(체력 포션)을 만들면서 C++ 매크로 시스템을 손으로 체감**
  - *(그 이후) 스테이지 데이터 테이블화 — 디자이너가 엑셀로 게임 만지기*

질문 받습니다